



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Evolución de un editor
Entidad-Relación con React y
mxGraph
Documentación Técnica**



Presentado por Steven Paul Alba Alba
en Universidad de Burgos — 24 de junio
de 2026

Tutor: Jesús Manuel Maudes Raedo

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	7
Apéndice B Especificación de Requisitos	13
B.1. Introducción	13
B.2. Objetivos generales	13
B.3. Catálogo de requisitos	15
B.4. Especificación de requisitos	18
Apéndice C Especificación de diseño	33
C.1. Introducción	33
C.2. Diseño de datos	33
C.3. Diseño procedimental	37
C.4. Diseño arquitectónico	40
Apéndice D Documentación técnica de programación	43
D.1. Introducción	43
D.2. Estructura de directorios	43
D.3. Manual del programador	43

D.4. Compilación, instalación y ejecución del proyecto	43
D.5. Pruebas del sistema	43
Apéndice E Documentación de usuario	45
E.1. Introducción	45
E.2. Requisitos de usuario	45
E.3. Instalación	46
E.4. Manual del usuario	47
Apéndice F Anexo de sostenibilización curricular	63
F.1. Introducción	63
F.2. Competencias de sostenibilidad adquiridas	63
F.3. Reflexión final	65
Bibliografía	67

Índice de figuras

A.1. Milestone 1: preparación del entorno y comprensión del proyecto.	3
A.2. Milestone 2: estabilización del núcleo y entidades débiles.	4
A.3. Milestone 3: refactorización, optimización y atributos avanzados.	4
A.4. Milestone 4: implementación de relaciones ternarias.	5
A.5. Milestone 5: soporte inicial de ISA y mejoras finales de usabilidad.	5
A.6. Cronograma resumido de los sprints adaptados del proyecto.	6
C.1. Estructura simplificada del modelo interno del diagrama.	34
C.2. Flujo de persistencia local e intercambio de diagramas mediante JSON.	36
C.3. Flujo de validación, transformación relacional y generación de SQL.	39
C.4. Arquitectura general de la aplicación.	41
E.1. Pantalla principal de UBU E-R App con un elemento seleccionado.	48
E.2. Componentes principales de la interfaz de usuario.	49
E.3. Configuración de los participantes de una relación.	52
E.4. Configuración de cardinalidades de una relación.	53
E.5. Resultado de la validación de un diagrama.	54
E.6. Script SQL generado a partir del diagrama.	55
E.7. Importación de un diagrama desde un fichero JSON.	56
E.8. Selección múltiple de elementos en el lienzo.	58
E.9. Información general del proyecto.	60

Índice de tablas

A.1. Estimación de horas dedicadas por bloque de trabajo.	8
A.2. Coste total estimado del proyecto.	9
A.3. Licencias principales consideradas en el proyecto.	10
B.1. CU-1 Crear y editar entidades.	19
B.2. CU-2 Crear y editar atributos.	20
B.3. CU-3 Configurar atributos especiales.	21
B.4. CU-4 Crear y configurar relaciones binarias.	22
B.5. CU-5 Crear y configurar relaciones ternarias y roles.	23
B.6. CU-6 Modelar entidades débiles y relaciones identificadoras.	24
B.7. CU-7 Modelar jerarquías ISA sencillas.	25
B.8. CU-8 Validar explícitamente el diagrama.	26
B.9. CU-9 Generar SQL a partir del diagrama.	27
B.10.CU-10 Guardar y recuperar el diagrama localmente.	28
B.11.CU-11 Exportar e importar diagramas en JSON.	29
B.12.CU-12 Generar estructuras básicas de ejemplo.	30
B.13.CU-13 Exportar la representación visual del diagrama.	31
B.14.CU-14 Utilizar acciones de apoyo de la interfaz.	32

Apéndice A

Plan de Proyecto Software

A.1. Introducción

En este anexo se describe la planificación seguida durante el desarrollo del Trabajo Fin de Grado, así como el estudio de viabilidad asociado. A diferencia de un proyecto iniciado desde cero, este trabajo parte de una aplicación heredada, por lo que la planificación no se centró únicamente en añadir nuevas funcionalidades, sino también en comprender el sistema existente, estabilizar su comportamiento y ampliar sus capacidades de forma controlada.

La planificación temporal recoge la forma en la que se organizó el trabajo, las fases principales del desarrollo y los mecanismos utilizados para realizar el seguimiento del proyecto. En este proceso se utilizaron issues y milestones de GitHub, ramas de desarrollo, revisiones con el tutor y despliegues funcionales en Vercel, que permitieron comprobar el estado de la aplicación desde el navegador sin depender de una instalación local.

El estudio de viabilidad analiza los costes que tendría un proyecto de estas características en un contexto profesional, así como las consideraciones legales relacionadas con las licencias de las herramientas y librerías utilizadas.

A.2. Planificación temporal

Metodología y organización del trabajo

El desarrollo del proyecto se organizó mediante una metodología ágil basada en Scrum, adaptada al contexto de un Trabajo Fin de Grado indivi-

dual. No se aplicó Scrum de forma completa, ya que no existía un equipo de desarrollo con roles diferenciados ni reuniones diarias, pero sí se utilizaron varios de sus principios: división del trabajo en bloques, definición de tareas concretas, revisión periódica de resultados y adaptación de la planificación según el avance del proyecto.

La organización práctica del trabajo se apoyó principalmente en GitHub. Los issues se utilizaron para registrar tareas concretas de análisis, implementación, pruebas, revisión y documentación. Estos issues actuaron como backlog del proyecto y permitieron mantener trazabilidad entre los objetivos planteados y el trabajo realizado.

Para facilitar su clasificación, se definió un conjunto de labels en GitHub. Estas etiquetas permitieron distinguir la naturaleza de cada tarea, por ejemplo si estaba relacionada con la configuración del entorno, el despliegue, la documentación, las pruebas, el modelo Entidad-Relación, mejoras funcionales, cuestiones teóricas o elementos centrales del TFG. Entre las etiquetas utilizadas se encontraban *setup*, *deployment*, *documentation*, *testing*, *er-model*, *enhancement*, *theory* y *tfg-core*.

Los milestones agruparon issues relacionados y se utilizaron como sprints adaptados. Cada milestone representó un bloque de trabajo de mayor duración que un sprint Scrum convencional, centrado en una parte concreta de la evolución del editor. Esta adaptación resultó adecuada para el proyecto, ya que se trataba de un TFG individual basado en una aplicación heredada y no de un desarrollo en equipo desde cero.

Además, se trabajó con ramas de desarrollo para separar cambios funcionales o de documentación. Los despliegues en Vercel permitieron publicar versiones funcionales de la aplicación y facilitaron su revisión desde el navegador, sin necesidad de instalar el proyecto en local.

En este contexto, la equivalencia entre Scrum y la organización real del TFG puede resumirse de la siguiente forma: los issues actuaron como tareas del backlog, las labels permitieron clasificar el trabajo, los milestones funcionaron como sprints adaptados, las versiones integradas y desplegadas como incrementos revisables, y las revisiones con el tutor como mecanismo de seguimiento y ajuste del trabajo.

Sprints adaptados del proyecto

El desarrollo se dividió en varios sprints adaptados, correspondientes a los milestones definidos en GitHub. A diferencia de una planificación Scrum

estricta, estos sprints no tuvieron una duración homogénea, sino que se ajustaron a la complejidad de cada bloque de trabajo y al avance real del proyecto.

Cada sprint adaptado agrupó issues relacionados con un objetivo común. Estos issues no se limitaron a la implementación de una funcionalidad aislada, sino que también incluyeron tareas de análisis, revisión del comportamiento, validación, pruebas, documentación y ajustes derivados de las revisiones realizadas.

El criterio seguido en todos los sprints fue mantener la coherencia entre el lienzo del editor, el modelo interno, las validaciones, la transformación al modelo relacional y la generación SQL. Por este motivo, estas comprobaciones se consideran transversales a cada milestone y no fases independientes del proyecto.

En las figuras incluidas a continuación se muestran los milestones correspondientes en GitHub, donde se recoge su descripción, estado de avance, fecha prevista e issues asociados.

Sprint adaptado 1: preparación del entorno y comprensión del proyecto

Este sprint se corresponde con el milestone *Environment setup and project understanding*. Su objetivo fue preparar el entorno de desarrollo y comprender el estado inicial de la aplicación heredada antes de introducir cambios funcionales relevantes. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

Durante esta fase se revisó la estructura del repositorio, se configuró la ejecución local, se comprobaron los comandos principales del proyecto y se realizó un primer despliegue en Vercel. El resultado fue disponer de una base de trabajo preparada y una organización inicial mediante issues, labels, milestones y ramas de desarrollo.

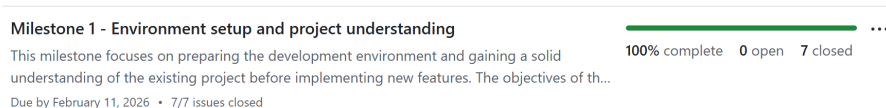


Figura A.1: Milestone 1: preparación del entorno y comprensión del proyecto.

Sprint adaptado 2: estabilización del núcleo y entidades débiles

Este sprint se corresponde con el milestone *Core stabilization and Weak Entities implementation*. Su objetivo fue estabilizar partes relevantes del núcleo del editor y comenzar la ampliación del modelo Entidad-Relación mediante el soporte de entidades débiles. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

El trabajo realizado permitió revisar la sincronización entre la representación visual y el modelo interno, así como integrar entidades débiles en el flujo de la aplicación. Esta ampliación se comprobó teniendo en cuenta su efecto sobre el lienzo, el modelo interno, las validaciones, la transformación relacional y la generación SQL.

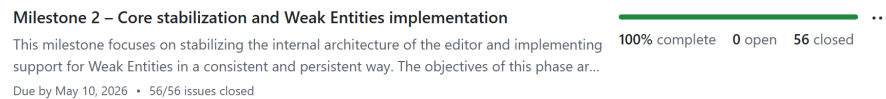


Figura A.2: Milestone 2: estabilización del núcleo y entidades débiles.

Sprint adaptado 3: refactorización, optimización y atributos avanzados

Este sprint se corresponde con el milestone *Code refactoring, optimization and advanced attributes implementation*. Su objetivo fue reducir la complejidad de algunas partes del sistema y ampliar el soporte de atributos del modelo Entidad-Relación. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

Durante este sprint se trabajó en la mejora de la estructura interna del editor y en el soporte de atributos avanzados, como atributos compuestos y multivaluados. Estos cambios se revisaron considerando su representación visual, almacenamiento interno, validaciones, transformación relacional y generación SQL.

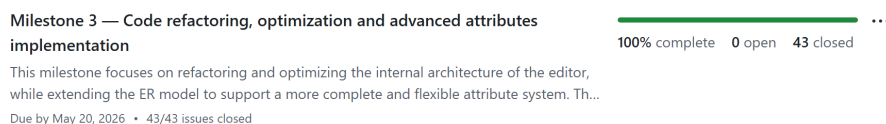


Figura A.3: Milestone 3: refactorización, optimización y atributos avanzados.

Sprint adaptado 4: implementación de relaciones ternarias

Este sprint se corresponde con el milestone *Ternary relationships implementation*. Su objetivo fue ampliar el editor para soportar relaciones ternarias, manteniendo el comportamiento existente de los elementos ya implementados. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

La implementación de relaciones ternarias afectó a varias partes del sistema: representación en el lienzo, participantes de la relación, roles, atributos propios, modelo interno, transformación relacional y SQL generado. El resultado fue la incorporación controlada de relaciones ternarias dentro del alcance del TFG.

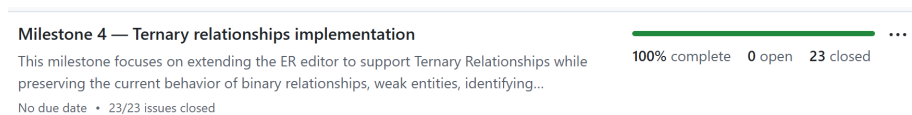


Figura A.4: Milestone 4: implementación de relaciones ternarias.

Sprint adaptado 5: soporte inicial de ISA y mejoras finales de usabilidad

Este sprint se corresponde con el milestone *ISA initial support and editor usability refinements*. Su objetivo fue introducir un soporte inicial para jerarquías ISA y cerrar mejoras de usabilidad e interacción solicitadas o detectadas durante la revisión del editor. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

El soporte de ISA se abordó de forma limitada y controlada, incluyendo generalización, especializaciones, clave heredada, transformación al modelo relacional y generación SQL asociada. Además, se incorporaron mejoras finales como ayuda integrada, información del proyecto, selección múltiple visible, operaciones sobre multiselección, exportación visual, ajuste de vista y opciones de importación. El resultado fue una versión más completa y usable de UBU E-R App, manteniendo explícitas las limitaciones de alcance.

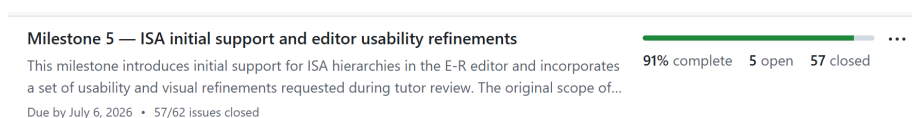


Figura A.5: Milestone 5: soporte inicial de ISA y mejoras finales de usabilidad.

Cronograma general

La Figura A.6 muestra una visión temporal resumida de los sprints adaptados del proyecto. Su finalidad no es representar actividades aisladas, sino situar en el tiempo los principales bloques funcionales abordados durante el desarrollo.

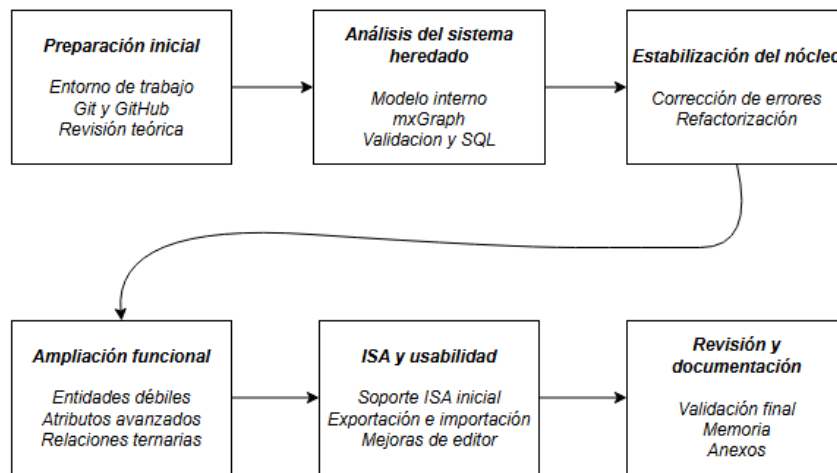


Figura A.6: Cronograma resumido de los sprints adaptados del proyecto.

Seguimiento y validación del proyecto

El seguimiento del proyecto se realizó a partir de los milestones e issues definidos en GitHub, junto con las revisiones mantenidas con el tutor. Estas revisiones permitieron comprobar el avance del trabajo, ajustar prioridades y detectar problemas de enfoque tanto en la aplicación como en la memoria y los anexos.

Los despliegues en Vercel fueron un apoyo importante en este proceso, ya que permitieron revisar versiones funcionales del editor directamente desde el navegador, sin necesidad de instalar el proyecto en local. De esta forma, los cambios integrados podían comprobarse sobre una versión accesible de la aplicación.

La validación técnica se realizó de forma progresiva durante el desarrollo. Algunas partes del proyecto se comprobaron mediante pruebas automáticas, especialmente aquellas relacionadas con validación, transformación relacional, generación SQL y comportamientos concretos del editor. Otras partes

requirieron revisión manual, sobre todo las relacionadas con la interacción visual en el lienzo, la sincronización entre la vista y el modelo interno, la importación y exportación de diagramas y el comportamiento general de la interfaz.

Esta validación no se concentró únicamente al final del proyecto. En cada sprint adaptado se revisó que las funcionalidades desarrolladas mantuvieran la coherencia entre el lienzo, el modelo interno, las validaciones, la transformación al modelo relacional y la generación SQL.

A.3. Estudio de viabilidad

El estudio de viabilidad permite valorar si el proyecto puede llevarse a cabo con los recursos disponibles y dentro del contexto académico en el que se desarrolla. En este caso, se analiza principalmente la viabilidad económica y la viabilidad legal.

Viabilidad económica

La viabilidad económica del proyecto puede analizarse desde dos puntos de vista. Por un lado, el coste directo real para el desarrollo del TFG fue reducido, ya que se utilizó un equipo propio y herramientas gratuitas o de acceso libre. Por otro lado, para valorar el esfuerzo necesario en un contexto profesional, se realiza una estimación económica del trabajo desarrollado.

El principal coste del proyecto corresponde al tiempo dedicado al análisis de la aplicación heredada, la implementación de mejoras, la revisión funcional, las pruebas y la elaboración de la documentación. Al tratarse de un proyecto basado en una aplicación existente, una parte importante del esfuerzo se destinó a comprender el sistema previo y a garantizar que las ampliaciones realizadas mantuvieran la coherencia entre el lienzo, el modelo interno, las validaciones, la transformación relacional y la generación SQL.

Costes de personal

Para estimar el coste de personal se ha tomado como referencia una dedicación total aproximada de 713 horas. Esta estimación incluye las tareas de preparación inicial, análisis del sistema heredado, implementación, validación, pruebas, revisión manual y documentación final.

La Tabla A.1 muestra una distribución aproximada de horas por bloques de trabajo.

Bloque de trabajo	Horas estimadas
Preparación del entorno y comprensión inicial del proyecto	65 h
Estabilización del núcleo y soporte de entidades débiles	205 h
Refactorización, optimización y atributos avanzados	145 h
Implementación de relaciones ternarias	115 h
Soporte inicial de ISA y mejoras finales de usabilidad	120 h
Memoria, anexos y revisión final	63 h
Total	713 h

Tabla A.1: Estimación de horas dedicadas por bloque de trabajo.

Para realizar la estimación económica se ha utilizado un coste orientativo de 15 €/h, correspondiente a una valoración prudente del trabajo de un perfil junior en un contexto académico. Con este criterio, el coste de personal estimado es el siguiente:

$$713 \text{ h} \times 15 \text{ €/h} = 10,695 \text{ €} \quad (\text{A.1})$$

Por tanto, el coste estimado de personal asciende a 10.695 €.

Costes de hardware

El desarrollo se realizó utilizando un ordenador de sobremesa propio, valorado aproximadamente en 1.200 €. Aunque no supuso un gasto directo específico para este TFG, se puede imputar una parte de su coste mediante amortización.

Para esta estimación se considera una vida útil de cuatro años, equivalente a 48 meses, y una duración aproximada del proyecto de cinco meses y medio. El coste imputado al proyecto se calcula de la siguiente forma:

$$\frac{1,200 \text{ €}}{48 \text{ meses}} \times 5,5 \text{ meses} = 137,50 \text{ €} \quad (\text{A.2})$$

Por tanto, el coste de hardware imputado al proyecto es de 137,50 €.

Costes de software y servicios

Las herramientas utilizadas durante el desarrollo no supusieron un coste económico directo. El proyecto se desarrolló con herramientas gratuitas o disponibles sin coste para el uso realizado, como GitHub, Vercel, Visual Studio Code, Node.js, npm, navegadores web y las librerías principales empleadas por la aplicación.

También se utilizaron herramientas de apoyo para pruebas, formateo, control de versiones y redacción de la memoria, sin necesidad de contratar planes de pago. Por tanto, el coste directo asociado al software y a los servicios empleados se considera nulo.

Coste total estimado

La Tabla A.2 resume el coste económico estimado del proyecto.

Concepto	Coste estimado
Coste de personal	10.695,00 €
Coste imputado de hardware	137,50 €
Costes de software y servicios	0,00 €
Costes adicionales	0,00 €
Total estimado	10.832,50 €

Tabla A.2: Coste total estimado del proyecto.

En conclusión, el proyecto fue viable económicamente dentro del contexto académico, ya que no requirió inversión directa en infraestructura ni contratación de servicios externos. La mayor parte del coste estimado corresponde al tiempo de desarrollo, análisis y documentación necesario para evolucionar una aplicación heredada de forma controlada.

Viabilidad legal

Desde el punto de vista legal, el proyecto se apoya en tecnologías y librerías de uso habitual en el desarrollo web. Al tratarse de la evolución de una aplicación existente, se revisó la licencia del proyecto original y las licencias de las principales dependencias utilizadas, con el objetivo de comprobar que la versión final pudiera distribuirse de forma coherente.

Licencias del software

La versión actual del proyecto se distribuye bajo licencia MIT. Esta licencia es adecuada para el contexto del TFG porque permite el uso, copia, modificación y distribución del código, manteniendo la obligación de conservar el aviso de copyright y el texto de la licencia.

Las dependencias principales utilizadas por la aplicación mantienen sus propias licencias. La Tabla A.3 resume las licencias más relevantes consideradas durante el desarrollo. En general, se trata de licencias permisivas, principalmente MIT [3] y Apache License 2.0 [4], compatibles con el uso académico y con la distribución del proyecto bajo licencia MIT.

Elemento	Licencia	Observación
Código desarrollado en el proyecto	MIT	La versión final incorpora un fichero <code>LICENSE</code> y la información correspondiente en <code>package.json</code> .
React	MIT	Librería principal empleada para construir la interfaz de usuario.
Material UI	MIT	Biblioteca de componentes utilizada para diferentes elementos de la interfaz.
mxGraph	Apache License 2.0	Librería utilizada para la creación y manipulación del lienzo de diagramas.
Vitest	MIT	Herramienta utilizada para la ejecución de pruebas unitarias.
Playwright	Apache License 2.0	Herramienta utilizada para la ejecución de pruebas end-to-end.
Biome	MIT	Herramienta empleada como apoyo para el análisis estático y el formateo del código.
Lefthook	MIT	Herramienta utilizada como apoyo en la automatización de comprobaciones durante el desarrollo.

Tabla A.3: Licencias principales consideradas en el proyecto.

Documentación, capturas y recursos gráficos

La memoria y los anexos se elaboran como documentación académica del Trabajo Fin de Grado. Las figuras utilizadas en la documentación corresponden principalmente a capturas propias de la aplicación, diagramas elaborados para explicar el proyecto o capturas de herramientas empleadas durante el desarrollo, como GitHub o Vercel.

En caso de incorporar recursos externos, estos deben citarse correctamente y respetar las condiciones de uso correspondientes. No obstante, el proyecto no depende de bancos de imágenes, vídeos de terceros ni datasets externos, por lo que los riesgos legales asociados a este tipo de recursos son reducidos.

Protección de datos

La aplicación funciona como herramienta cliente y no incorpora registro de usuarios, autenticación, backend propio ni envío de diagramas a un servidor. El almacenamiento del diagrama se realiza en el navegador del usuario o mediante archivos exportados cuando el propio usuario decide guardarlos.

Por este motivo, el proyecto no plantea un tratamiento centralizado de datos personales. Aun así, los diagramas creados o importados deben considerarse información bajo control del usuario, ya que pueden contener nombres o datos introducidos libremente durante el uso de la herramienta.

En conclusión, el proyecto resulta viable desde el punto de vista legal. Las licencias principales son compatibles con el uso académico y con la distribución del código bajo licencia MIT, no se utilizan recursos externos críticos y la aplicación no incorpora tratamiento remoto de datos personales.

Apéndice B

Especificación de Requisitos

B.1. Introducción

En este apartado se enumeran y desarrollan los objetivos y requisitos que debe tener la aplicación según el alcance final del proyecto. La herramienta parte de un editor de diagramas Entidad-Relación ya existente, por lo que los requisitos no se plantean como si el sistema se hubiera desarrollado desde cero, sino como una evolución de una aplicación heredada.

Durante el trabajo se han incorporado nuevas funcionalidades del modelo Entidad-Relación, se ha revisado la validación y la generación de SQL, y se han añadido mejoras de usabilidad para facilitar la edición de diagramas. Por ello, este anexo recoge tanto las funcionalidades principales del editor como las ampliaciones realizadas durante el desarrollo del TFG.

B.2. Objetivos generales

Los objetivos generales del proyecto se pueden resumir en los siguientes puntos:

- Modelar diagramas E-R mediante una aplicación web.
- Almacenar los diagramas en una estructura interna coherente y extensible.
- Representar elementos básicos y avanzados del modelo E-R dentro del alcance implementado.
- Validar la estructura interna del diagrama para detectar errores antes de generar resultados.

- Transformar el diagrama E-R validado a una representación relacional.
- Generar scripts SQL a partir de la representación relacional obtenida.
- Exportar e importar diagramas en formato JSON.
- Permitir la combinación controlada de diagramas importados o estructuras generadas.
- Exportar la representación visual del diagrama en formatos de imagen.
- Mejorar la usabilidad del editor sin alterar la notación académica del modelo E-R, incorporando acciones contextuales, selección múltiple visible y secciones de ayuda e información del proyecto.

B.3. Catálogo de requisitos

En este apartado se definirán los requisitos funcionales y no funcionales del proyecto.

Requisitos funcionales

- **RF1 - Modelar elementos básicos E-R:** La aplicación debe permitir crear y manipular entidades, atributos y relaciones dentro del lienzo del editor.
- **RF2 - Configurar atributos:** La aplicación debe permitir trabajar con atributos simples, clave, compuestos, multivaluados y discriminantes de entidades débiles en los casos soportados.
- **RF3 - Configurar relaciones:** La aplicación debe permitir configurar relaciones binarias, cardinalidades y atributos propios de relación cuando proceda.
- **RF4 - Modelar relaciones ternarias y roles:** La aplicación debe permitir representar relaciones de grado tres y distinguir roles de participación cuando sean necesarios.
- **RF5 - Modelar entidades débiles:** La aplicación debe permitir representar entidades débiles, relaciones identificadoras y discriminantes.
- **RF6 - Modelar jerarquías ISA sencillas:** La aplicación debe permitir representar una generalización con una o varias especializaciones.
- **RF7 - Controlar el alcance de ISA:** La aplicación debe acotar el soporte de ISA al caso implementado. En particular, no debe tratar especializaciones con clave propia, herencia múltiple real, discriminantes ISA ni restricciones de totalidad, parcialidad, solapamiento o exclusividad.
- **RF8 - Mantener el modelo interno:** La aplicación debe mantener una estructura interna coherente con el contenido representado en el lienzo.
- **RF9 - Persistir diagramas localmente:** La aplicación debe conservar el diagrama en el almacenamiento local del navegador.
- **RF10 - Exportar e importar JSON:** La aplicación debe permitir exportar diagramas a JSON e importar diagramas desde archivos JSON válidos.
- **RF11 - Reemplazar o combinar contenido:** La aplicación debe permitir importar diagramas o generar estructuras reemplazando el diagrama actual o combinándolas con el contenido existente.

- **RF12 - Validar diagramas:** La aplicación debe validar el modelo interno y mostrar los errores detectados de forma comprensible.
- **RF13 - Generar SQL:** La aplicación debe transformar el diagrama validado al modelo relacional y generar un script SQL con un formato claro y legible.
- **RF14 - Generar estructuras básicas:** La aplicación debe permitir crear estructuras de ejemplo para facilitar la prueba y exploración del editor.
- **RF15 - Exportar imagen del diagrama:** La aplicación debe permitir exportar la representación visual del diagrama en formato PNG y SVG.
- **RF16 - Ofrecer acciones de apoyo:** La aplicación debe incluir acciones como comprobar el diagrama, ajustar la vista, cambiar el idioma de la interfaz, consultar ayuda, visualizar información del proyecto y reiniciar el lienzo previa confirmación.
- **RF17 - Gestionar selección múltiple y acciones contextuales:** La aplicación debe permitir seleccionar varios elementos de forma visible, mover o eliminar una selección múltiple y evitar mostrar acciones individuales cuando hay varios elementos seleccionados.

El soporte de ISA se considera inicial y controlado: se contempla una única generalización con una o varias especializaciones, con clave heredada desde la generalización, pero quedan fuera del alcance la totalidad o parcialidad, el solapamiento o exclusividad, los discriminantes ISA y la herencia múltiple real. Las agregaciones no forman parte de los requisitos implementados en esta versión.

Requisitos no funcionales

- **RNF1 - Usabilidad:** La aplicación debe ofrecer una interfaz clara y cómoda para la creación y revisión de diagramas E-R, incluyendo guías, descripciones breves de acciones, acciones contextuales y retroalimentación visual durante la selección de elementos.
- **RNF2 - Compatibilidad:** La aplicación debe poder ejecutarse en navegadores web modernos.
- **RNF3 - Mantenibilidad:** El código debe organizarse de forma que permita seguir ampliando el editor con nuevas funcionalidades.
- **RNF4 - Coherencia visual:** La representación gráfica debe respetar la notación académica utilizada para entidades, relaciones, atributos, entidades débiles, relaciones identificadoras e ISA.
- **RNF5 - Separación de responsabilidades:** La aplicación debe mantener una separación razonable entre interfaz, modelo interno, validación, transformación relacional y generación SQL.
- **RNF6 - Funcionamiento sin backend:** La aplicación debe funcionar sin requerir autenticación de usuarios, servidor propio ni almacenamiento remoto.
- **RNF7 - Contexto académico:** La herramienta debe servir como apoyo práctico para el aprendizaje y uso del modelo Entidad-Relación y su transformación relacional.

B.4. Especificación de requisitos

Los requisitos funcionales se agrupan en varios bloques. El primero corresponde a la edición visual del diagrama, que incluye la creación y modificación de entidades, atributos y relaciones. El segundo bloque recoge los elementos avanzados del modelo E-R soportados por la aplicación, como entidades débiles, atributos compuestos y multivaluados, relaciones ternarias, roles y jerarquías ISA sencillas.

Otro bloque importante es el mantenimiento del modelo interno. Cada elemento representado en el lienzo debe conservarse en una estructura de datos que permita reconstruir el diagrama, validarlo, persistirlo y transformarlo posteriormente. Esta estructura es también la base para la exportación e importación en formato JSON.

La validación del diagrama permite comprobar si el modelo contiene errores antes de generar una salida relacional o SQL. Además, el usuario puede ejecutar una comprobación explícita del diagrama sin necesidad de generar SQL. Si se detectan errores, la aplicación debe mostrarlos de forma agrupada y comprensible.

La transformación relacional y la generación de SQL constituyen la salida principal del sistema. A partir del diagrama validado, la aplicación obtiene una representación basada en tablas, columnas, claves primarias y claves foráneas, y posteriormente genera el script SQL correspondiente.

Finalmente, la aplicación incorpora varias funcionalidades de apoyo a la edición: generación de estructuras básicas, importación y generación combinada de contenido, exportación visual en PNG y SVG, ajuste de la vista, soporte básico de idioma, selección múltiple visible, acciones contextuales, secciones de ayuda e información del proyecto y reinicio controlado del lienzo.

Casos de uso

Las tablas siguientes describen los principales casos de uso del sistema a nivel funcional, sin entrar en detalles internos de implementación ni en pasos concretos dependientes de la interfaz.

CU-1	Crear y editar entidades
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF1, RF8
Descripción	Permite al usuario crear entidades en el lienzo y modificar sus propiedades básicas.
Precondición	La aplicación debe estar abierta y funcionando.
Acciones	1. El usuario crea una entidad en el lienzo. 2. El usuario puede mover, seleccionar, renombrar o eliminar la entidad. 3. Si se trata de una entidad fuerte normal, la aplicación puede crear una clave <code>id</code> por defecto.
Postcondición	La entidad queda representada visualmente y almacenada en el modelo interno.
Excepciones	-
Importancia	Alta

Tabla B.1: CU-1 Crear y editar entidades.

CU-2	Crear y editar atributos
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF1, RF2, RF8
Descripción	Permite añadir atributos a entidades o relaciones y modificar su información básica.
Precondición	Debe existir una entidad o relación seleccionable en el diagrama.
Acciones	1. El usuario selecciona el elemento al que desea añadir un atributo. 2. El usuario crea el atributo. 3. El usuario puede renombrar, mover o eliminar el atributo.
Postcondición	El atributo queda asociado al elemento correspondiente en el lienzo y en el modelo interno.
Excepciones	El atributo no se crea si no existe un elemento válido al que asociarlo.
Importancia	Alta

Tabla B.2: CU-2 Crear y editar atributos.

CU-3	Configurar atributos especiales
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF2
Descripción	Permite configurar atributos con significado especial dentro del modelo E-R.
Precondición	Debe existir al menos un atributo asociado a una entidad o relación.
Acciones	1. El usuario selecciona un atributo. 2. El usuario configura el atributo como clave, discriminante de entidad débil, compuesto o multi-valuado cuando la acción está disponible. 3. En atributos compuestos, el usuario puede añadir subatributos.
Postcondición	El atributo queda representado con la notación correspondiente y su tipo se conserva en el modelo interno.
Excepciones	Algunas acciones no están disponibles si el atributo pertenece a un elemento donde esa configuración no es válida.
Importancia	Alta

Tabla B.3: CU-3 Configurar atributos especiales.

CU-4	Crear y configurar relaciones binarias
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF3, RF8
Descripción	Permite crear relaciones binarias entre entidades y configurar sus participantes y cardinalidades.
Precondición	Deben existir entidades en el diagrama.
Acciones	1. El usuario crea una relación. 2. El usuario selecciona las entidades participantes. 3. El usuario configura las cardinalidades de la relación. 4. Si procede, el usuario añade atributos propios a la relación.
Postcondición	La relación queda conectada a sus entidades participantes y almacenada en el modelo interno.
Excepciones	La relación puede quedar pendiente de validación si faltan participantes o cardinalidades.
Importancia	Alta

Tabla B.4: CU-4 Crear y configurar relaciones binarias.

CU-5	Crear y configurar relaciones ternarias y roles
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF4, RF8
Descripción	Permite representar relaciones de grado tres y distinguir roles de participación cuando sea necesario.
Precondición	Deben existir entidades suficientes para configurar una relación ternaria.
Acciones	1. El usuario crea una relación ternaria. 2. El usuario selecciona sus tres participantes. 3. El usuario configura cardinalidades. 4. El usuario define roles cuando una entidad participa más de una vez o cuando se requiere mayor claridad.
Postcondición	La relación ternaria queda almacenada con sus participantes, cardinalidades y roles.
Excepciones	La validación informa de participantes, cardinalidades o roles incorrectos cuando proceda.
Importancia	Alta

Tabla B.5: CU-5 Crear y configurar relaciones ternarias y roles.

CU-6	Modelar entidades débiles y relaciones identificadoras
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF5, RF8, RF13
Descripción	Permite representar una entidad débil y su dependencia por identificación respecto a una entidad propietaria.
Precondición	Deben existir las entidades necesarias para configurar la dependencia.
Acciones	1. El usuario marca una entidad como débil. 2. El usuario configura la relación identificadora. 3. El usuario establece la entidad propietaria. 4. El usuario define el discriminante correspondiente.
Postcondición	La entidad débil queda representada con la notación adecuada y se tiene en cuenta en la transformación relacional.
Excepciones	La validación informa si la entidad débil no tiene propietario, relación identificadora o discriminante válido.
Importancia	Alta

Tabla B.6: CU-6 Modelar entidades débiles y relaciones identificadoras.

CU-7	Modelar jerarquías ISA sencillas
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF6, RF7, RF8, RF13
Descripción	Permite representar una jerarquía ISA con una entidad general y una o varias especializaciones.
Precondición	Deben existir las entidades que participarán en la jerarquía.
Acciones	1. El usuario crea un elemento ISA. 2. El usuario selecciona la entidad general. 3. El usuario añade una o varias entidades especializadas. 4. La aplicación impide configuraciones fuera del alcance implementado.
Postcondición	La jerarquía ISA queda representada en el lienzo y almacenada en el modelo interno.
Excepciones	La validación informa de ISA sin generalización, sin especializaciones, con claves propias en especializaciones o con configuraciones de herencia no soportadas. Las restricciones de totalidad o parcialidad, solapamiento o exclusividad y discriminante ISA quedan fuera del alcance.
Importancia	Alta

Tabla B.7: CU-7 Modelar jerarquías ISA sencillas.

CU-8	Validar explícitamente el diagrama
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF12
Descripción	Permite comprobar si el diagrama actual es válido sin generar SQL ni exportar información.
Precondición	La aplicación debe estar abierta y puede existir un diagrama completo o en construcción.
Acciones	1. El usuario selecciona la acción de comprobar el diagrama. 2. La aplicación ejecuta las reglas de validación sobre el modelo interno. 3. La aplicación muestra el resultado de la validación.
Postcondición	Si el diagrama es válido, se muestra una notificación de éxito. Si contiene errores, se abre el diálogo de validación.
Excepciones	-
Importancia	Alta

Tabla B.8: CU-8 Validar explícitamente el diagrama.

CU-9	Generar SQL a partir del diagrama
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF12, RF13
Descripción	Permite generar un script SQL a partir del diagrama E-R validado.
Precondición	El diagrama debe contener una estructura suficiente para poder validarse.
Acciones	1. El usuario selecciona la opción de generar SQL. 2. La aplicación valida el diagrama. 3. La aplicación transforma el modelo E-R a una representación relacional. 4. La aplicación genera el script SQL correspondiente.
Postcondición	Si el diagrama es válido, se obtiene el SQL generado. Si no lo es, se muestran los errores detectados.
Excepciones	Errores de validación del diagrama.
Importancia	Alta

Tabla B.9: CU-9 Generar SQL a partir del diagrama.

CU-10	Guardar y recuperar el diagrama localmente
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF8, RF9
Descripción	Permite conservar el estado del diagrama entre recargas o sesiones del navegador.
Precondición	El usuario debe haber creado o modificado un diagrama.
Acciones	1. La aplicación mantiene actualizado el modelo interno. 2. La aplicación guarda el estado en el almacenamiento local del navegador. 3. Al recargar, la aplicación reconstruye el diagrama guardado.
Postcondición	El usuario puede continuar trabajando sobre el diagrama sin perder el contenido por una recarga.
Excepciones	El contenido puede no recuperarse si se borra el almacenamiento local del navegador.
Importancia	Alta

Tabla B.10: CU-10 Guardar y recuperar el diagrama localmente.

CU-11	Exportar e importar diagramas en JSON
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF10, RF11
Descripción	Permite guardar el diagrama en un archivo JSON o cargar un diagrama previamente exportado.
Precondición	Para exportar, debe existir un diagrama. Para importar, el usuario debe disponer de un archivo JSON compatible.
Acciones	1. El usuario selecciona la opción de exportar o importar JSON. 2. En la importación, el usuario elige entre reemplazar el diagrama actual o combinar el contenido importado. 3. La aplicación valida y reconstruye el resultado cuando procede.
Postcondición	El diagrama se guarda como JSON o se carga en el editor manteniendo una estructura interna coherente.
Excepciones	Archivo incompatible, JSON inválido o diagrama combinado no válido.
Importancia	Media

Tabla B.11: CU-11 Exportar e importar diagramas en JSON.

CU-12	Generar estructuras básicas de ejemplo
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF11, RF14
Descripción	Permite crear de forma rápida estructuras habituales del modelo E-R.
Precondición	La aplicación debe estar abierta y funcionando.
Acciones	1. El usuario abre la acción de generar estructura. 2. El usuario selecciona una estructura básica. 3. El usuario decide si desea reemplazar el diagrama actual o combinar la estructura con el contenido existente.
Postcondición	La estructura seleccionada aparece en el lienzo y queda integrada en el modelo interno.
Excepciones	La operación no se realiza si el usuario cancela la confirmación. Si el diagrama resultante queda incompleto o incoherente, podrá detectarse posteriormente mediante la validación del diagrama.
Importancia	Media

Tabla B.12: CU-12 Generar estructuras básicas de ejemplo.

CU-13	Exportar la representación visual del diagrama
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF15
Descripción	Permite obtener una imagen del diagrama para documentación, revisión o uso externo.
Precondición	Debe existir contenido visual en el lienzo del editor.
Acciones	1. El usuario selecciona la opción de exportar imagen. 2. El usuario elige el formato disponible, PNG o SVG. 3. La aplicación genera el archivo visual correspondiente.
Postcondición	El usuario obtiene una imagen del diagrama. Esta operación no modifica el modelo interno ni el SQL generado.
Excepciones	La aplicación informa si no existe contenido suficiente para exportar.
Importancia	Media

Tabla B.13: CU-13 Exportar la representación visual del diagrama.

CU-14	Utilizar acciones de apoyo de la interfaz
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF16, RF17
Descripción	Permite utilizar acciones auxiliares que facilitan la interacción con el editor.
Precondición	La aplicación debe estar abierta.
Acciones	1. El usuario puede ajustar la vista al contenido del diagrama. 2. El usuario puede cambiar el idioma de la interfaz entre español e inglés. 3. El usuario puede consultar la guía mostrada cuando no hay selección. 4. El usuario puede reiniciar el lienzo previa confirmación. 5. El usuario puede consultar la ayuda de uso y la información del proyecto. 6. El usuario puede seleccionar varios elementos de forma visible. 7. El usuario puede mover o eliminar una selección múltiple cuando proceda.
Postcondición	La edición del diagrama resulta más cómoda sin modificar la semántica del modelo E-R.
Excepciones	El reinicio del lienzo no se realiza si el usuario cancela la confirmación.
Importancia	Media

Tabla B.14: CU-14 Utilizar acciones de apoyo de la interfaz.

Apéndice C

Especificación de diseño

C.1. Introducción

En este anexo se describe el diseño general de la aplicación desarrollada durante el proyecto. El objetivo no es recoger todos los detalles internos del código, sino explicar cómo se organiza el editor y cómo se relacionan sus partes principales.

La aplicación parte de un proyecto heredado, por lo que el diseño final es el resultado de comprender una base existente y adaptarla progresivamente. El trabajo no consistió únicamente en añadir elementos visuales al lienzo, sino en mantener la coherencia entre la representación gráfica, el modelo interno, la validación, la transformación relacional y la generación de SQL.

De forma general, el sistema puede entenderse como una aplicación web compuesta por tres bloques principales. El primero es la interfaz visual, que permite al usuario crear y modificar el diagrama. El segundo es el modelo interno, que almacena la información estructurada del diagrama. El tercero está formado por los procesos que trabajan sobre ese modelo: validación, transformación relacional, generación de SQL, persistencia, importación y exportación.

C.2. Diseño de datos

El diseño de datos se centra en la estructura interna utilizada por la aplicación para representar los diagramas Entidad-Relación. Aunque el usuario trabaja sobre un lienzo visual, la aplicación necesita mantener

una representación propia del diagrama para poder guardarlo, validarlo, transformarlo y reconstruirlo posteriormente.

Modelo interno del diagrama

El modelo interno actúa como la fuente principal de información lógica del editor. En él se almacenan los elementos creados por el usuario, sus propiedades y las relaciones entre ellos. Esta representación permite que las operaciones principales de la aplicación no dependan únicamente de la posición o de la apariencia de los elementos gráficos.

La Figura C.1 muestra una visión simplificada de los principales elementos que forman parte del modelo interno del diagrama.

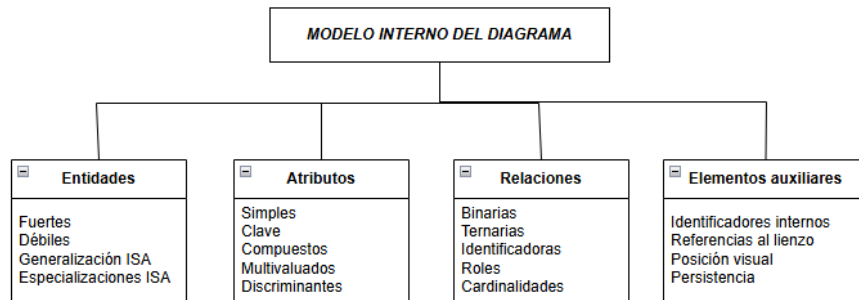


Figura C.1: Estructura simplificada del modelo interno del diagrama.

Cada elemento del diagrama tiene un identificador interno y una serie de propiedades asociadas. Esta información permite distinguir entidades, atributos, relaciones y elementos especiales, así como reconstruir el diagrama cuando se carga desde el almacenamiento local o desde un archivo JSON.

Entidades y atributos

Las entidades almacenan la información necesaria para identificarlas dentro del diagrama y para distinguir comportamientos especiales. Una entidad puede actuar como entidad fuerte, entidad débil, generalización o especialización dentro de una jerarquía ISA, según el caso.

Los atributos se asocian a entidades o relaciones. El modelo debe conservar su nombre, su elemento propietario y las propiedades que afectan a la notación y a la transformación posterior. Entre estas propiedades se incluyen los atributos clave, compuestos, multivaluados y discriminantes en los casos soportados.

En las entidades fuertes nuevas, la aplicación puede crear una clave `id` por defecto. Este comportamiento se aplica de forma controlada para no alterar la semántica de las entidades débiles ni de las especializaciones ISA, que tienen restricciones propias sobre sus claves.

Relaciones

Las relaciones almacenan sus participantes y las cardinalidades asociadas. Esta información es necesaria para validar el diagrama y para decidir cómo se transforma posteriormente al modelo relacional.

El modelo también debe permitir representar relaciones con atributos propios, relaciones identificadoras, relaciones ternarias y roles. En estos casos, la relación no solo une entidades, sino que contiene información adicional que afecta a la interpretación del diagrama y a la generación de la salida relacional.

Jerarquías ISA

El soporte de ISA se diseñó con un alcance inicial y controlado. El modelo permite representar una generalización con una o varias especializaciones. La entidad general mantiene su clave, mientras que las especializaciones heredan esa clave y no pueden definir claves propias.

Este diseño evita introducir variantes más complejas del modelo ISA, como totalidad o parcialidad, solapamiento o exclusividad, discriminantes de especialización o herencia múltiple real. Estas características quedan fuera del alcance implementado y se consideran posibles líneas de trabajo futuro.

En la transformación relacional, la generalización se convierte en una tabla ordinaria. Cada especialización se transforma en una tabla propia cuya clave primaria es la clave heredada de la generalización y, al mismo tiempo, una clave foránea hacia la tabla general [1].

Persistencia e intercambio de diagramas

El modelo interno también se utiliza para conservar el estado del diagrama. La aplicación guarda la información en el almacenamiento local del navegador, permitiendo que el usuario recupere el contenido al recargar la página.

Además, el sistema permite exportar e importar diagramas en formato JSON. La exportación genera una representación del modelo interno que puede almacenarse en un archivo. La importación permite reconstruir un

diagrama previamente guardado. En las operaciones de importación y generación de estructuras, el usuario puede reemplazar el contenido actual o combinarlo con el diagrama existente mediante una inserción controlada.

La Figura C.2 resume el papel del modelo interno como punto común entre la edición visual, la persistencia local y el intercambio de diagramas mediante JSON.

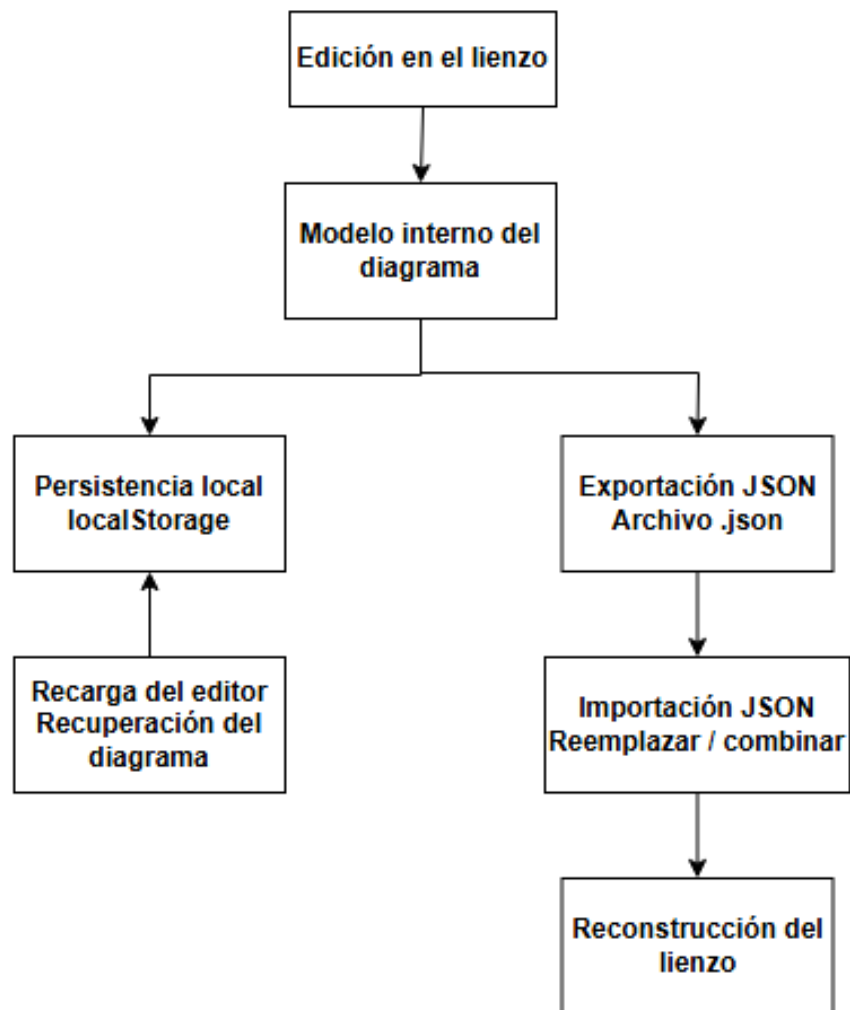


Figura C.2: Flujo de persistencia local e intercambio de diagramas mediante JSON.

C.3. Diseño procedimental

El diseño procedimental describe los principales flujos de funcionamiento de la aplicación. Estos flujos muestran cómo las acciones del usuario se convierten en cambios del modelo interno y cómo ese modelo se valida y transforma para generar resultados.

Flujo general de edición

El flujo principal comienza cuando el usuario crea o modifica elementos en el lienzo del editor. Cada acción visual debe reflejarse en el modelo interno, de forma que lo que aparece en pantalla coincida con la información que la aplicación conserva y procesa.

Este flujo incluye operaciones como crear entidades, añadir atributos, configurar relaciones, establecer cardinalidades, mover elementos, renombrarlos o eliminarlos. En todos los casos, la aplicación debe mantener la sincronización entre la vista y el modelo interno.

Además, la interfaz contempla la selección múltiple de elementos. Este comportamiento permite mover o eliminar varios elementos de forma conjunta, pero evita mostrar acciones individuales cuando la selección contiene más de un elemento. De esta forma, se mantiene la coherencia entre las acciones disponibles en la interfaz y las operaciones que realmente pueden aplicarse sobre el modelo.

Validación del modelo

Antes de generar una salida relacional o SQL, la aplicación valida el modelo interno. Esta validación comprueba que el diagrama contiene la información necesaria y que no existen configuraciones incompletas o incompatibles con el alcance implementado.

La validación se ejecuta en operaciones como la generación de SQL y también puede lanzarse de forma explícita mediante la acción de comprobar el diagrama. Cuando se detectan errores, la aplicación debe mostrarlos de forma comprensible para que el usuario pueda corregir el modelo.

En el caso de ISA, la validación impide configuraciones fuera del alcance implementado, como una ISA sin generalización, una ISA sin especializaciones, especializaciones con clave propia o entidades especializadas en más de una jerarquía.

Transformación al modelo relacional

Si el diagrama supera la validación, la aplicación transforma el modelo Entidad-Relación a una representación relacional. Esta representación se basa en tablas, columnas, claves primarias y claves foráneas.

La existencia de una representación intermedia permite separar la lógica de transformación de la generación concreta del script SQL. De esta forma, la aplicación primero decide qué tablas y restricciones deben existir, y después construye el texto SQL correspondiente.

Generación de SQL

La generación de SQL toma como entrada la representación relacional obtenida en la fase anterior. A partir de ella, se generan las sentencias necesarias para crear las tablas y definir sus claves principales y foráneas.

En la versión final se opta por una salida SQL más legible, evitando nombres explícitos de restricciones de clave foránea cuando no resultan necesarios y simplificando las referencias cuando una clave foránea referencia la clave primaria completa de la tabla destino. Esta decisión no modifica la transformación relacional previa, sino únicamente la forma en la que se renderiza el script SQL generado.

La salida SQL debe entenderse como una traducción razonable del modelo relacional generado por la aplicación, no como un generador industrial completo para cualquier sistema gestor de bases de datos. Su objetivo principal dentro del proyecto es servir como apoyo académico y como punto de partida para comprender la transformación desde el modelo Entidad-Relación.

La Figura C.3 resume este proceso: el modelo interno se valida antes de construir la representación relacional y solo después se genera el script SQL correspondiente.

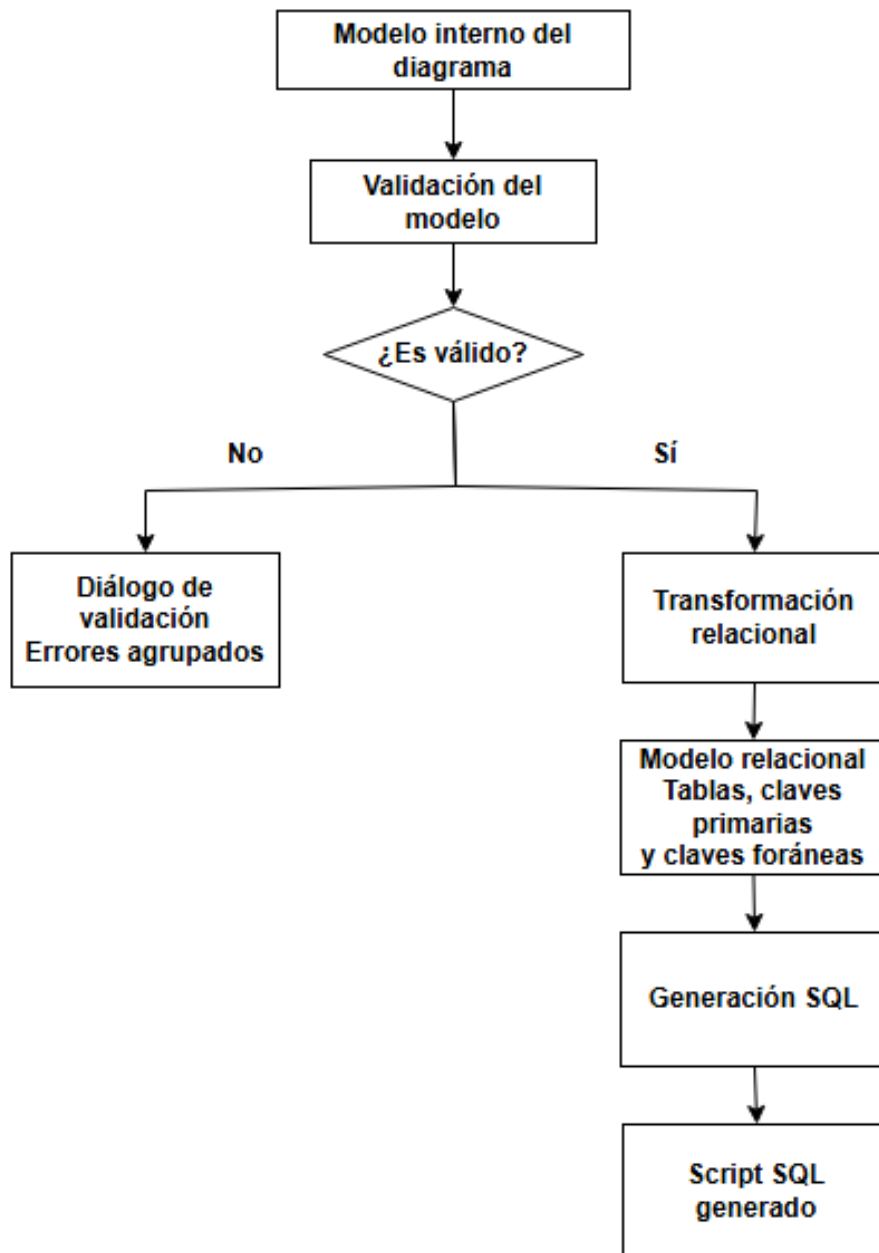


Figura C.3: Flujo de validación, transformación relacional y generación de SQL.

Importación, combinación y generación de estructuras

Además del flujo principal de edición, la aplicación incluye operaciones para importar diagramas JSON y generar estructuras básicas. En ambos

casos, el usuario puede reemplazar el diagrama actual o combinar el nuevo contenido con el existente.

Cuando se combina contenido, la aplicación debe evitar conflictos directos de identificadores y reducir posibles conflictos de nombres. Para ello, el proceso de inserción debe remapear referencias internas y desplazar visualmente los elementos añadidos, de forma que el resultado sea más claro para el usuario.

Estas operaciones se diseñan como ayudas a la edición, no como mecanismos de fusión semántica avanzada. Por tanto, la aplicación no intenta interpretar si dos entidades representan el mismo concepto, sino insertar el contenido de forma controlada.

C.4. Diseño arquitectónico

El diseño arquitectónico describe la organización general de la aplicación y la separación entre sus bloques principales. La herramienta se desarrolla como una aplicación web en la que la interfaz y la lógica principal se ejecutan en el cliente.

Arquitectura general

La aplicación está construida alrededor de un editor visual que permite manipular diagramas Entidad-Relación. La parte gráfica se apoya en un lienzo de edición, mientras que la lógica de la aplicación trabaja con el modelo interno del diagrama.

La separación entre vista y modelo es una de las decisiones más importantes del diseño, aunque en la práctica existe una dependencia entre el lienzo gráfico y el modelo interno durante la sincronización de elementos. La vista permite que el usuario interactúe con el diagrama, pero las operaciones principales intentan apoyarse en una representación estructurada del modelo. Gracias a ello, el sistema puede validar, persistir, importar, exportar y transformar diagramas sin depender únicamente de la representación visual.

La Figura C.4 resume la organización general de la aplicación, separando la interfaz visual, el modelo interno y los procesos que trabajan sobre dicho modelo.

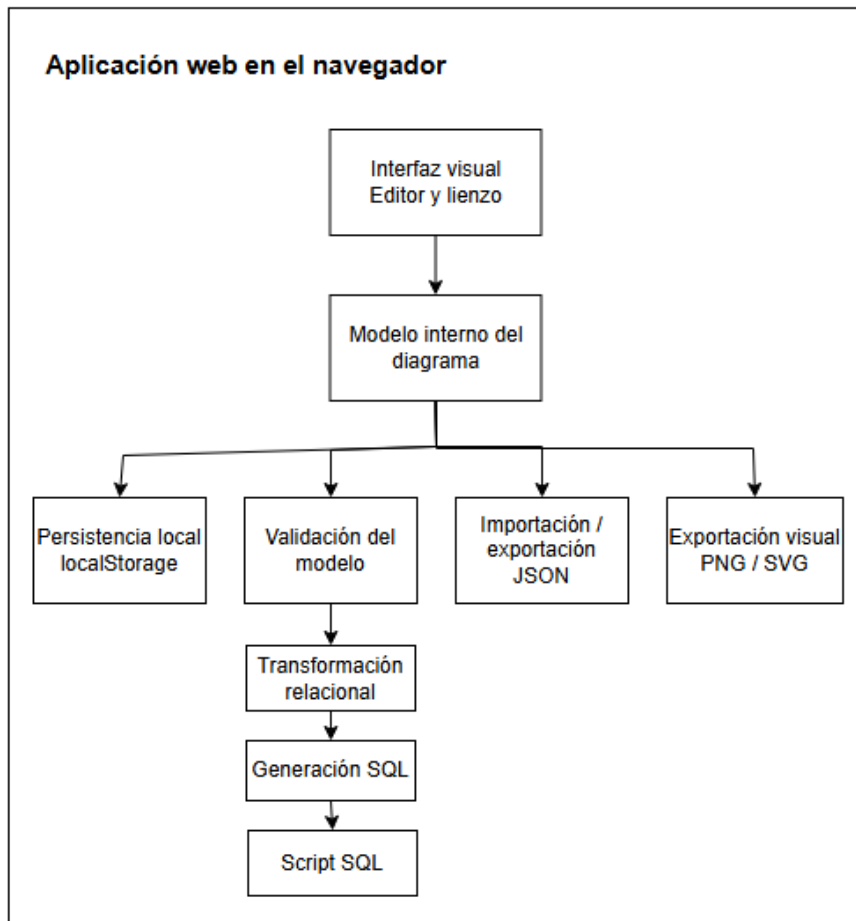


Figura C.4: Arquitectura general de la aplicación.

Organización por módulos

A nivel funcional, pueden distinguirse varios bloques dentro de la aplicación. El primero corresponde a la interfaz de usuario, encargada de mostrar el lienzo, los paneles de acciones, las acciones contextuales, los diálogos, los mensajes de validación y las secciones de ayuda e información del proyecto.

El segundo bloque corresponde al modelo interno del diagrama. Este bloque representa la información lógica creada por el usuario y sirve como base para el resto de operaciones. Sobre este modelo trabajan los procesos de validación, persistencia, transformación relacional y generación de SQL.

El tercer bloque está formado por las operaciones de entrada y salida. Aquí se incluyen la persistencia local, la exportación e importación de JSON, la generación de estructuras básicas y la exportación visual del diagrama en PNG y SVG.

Por último, la aplicación cuenta con módulos relacionados con la validación y transformación. Estos módulos comprueban que el diagrama sea coherente dentro del alcance implementado y generan una representación relacional que posteriormente se convierte en SQL.

Pruebas y revisión

El diseño del proyecto también tiene en cuenta la necesidad de revisar el comportamiento de la aplicación. Para ello se utilizan pruebas automáticas y comprobaciones manuales sobre casos representativos.

Las pruebas automáticas ayudan a detectar regresiones en funcionalidades concretas, especialmente en aquellas relacionadas con la validación, la transformación relacional, la generación de SQL y la persistencia. La revisión manual complementa estas pruebas, ya que permite comprobar aspectos visuales y de interacción que son difíciles de cubrir completamente mediante tests.

En conjunto, la arquitectura del sistema busca mantener una separación razonable entre la interfaz, el modelo interno y los procesos de validación y transformación. Esta organización facilita que el editor pueda seguir ampliándose en el futuro sin que cada nueva funcionalidad dependa exclusivamente de la representación gráfica.

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

E.1. Introducción

Este anexo presenta una guía de uso de la aplicación desarrollada. Su objetivo es explicar, desde el punto de vista del usuario final, cómo acceder al editor, qué requisitos básicos son necesarios y cómo utilizar las principales funcionalidades disponibles para crear, validar, importar, exportar y transformar diagramas Entidad-Relación.

A diferencia de la documentación técnica de programación, este apartado no se centra en la estructura interna del código ni en las decisiones de implementación. El propósito es ofrecer una referencia práctica para manejar la aplicación de forma autónoma, describiendo las acciones más habituales sobre el lienzo y los elementos del diagrama.

La aplicación puede utilizarse directamente desde el navegador y está orientada principalmente a usuarios con conocimientos básicos del modelo Entidad-Relación. Por ello, el manual asume que el usuario conoce los conceptos generales de entidad, atributo, relación, cardinalidad y clave primaria, aunque la propia interfaz incorpora mensajes de ayuda, acciones contextuales y validaciones para facilitar el uso del editor.

E.2. Requisitos de usuario

Para utilizar esta aplicación, los usuarios deben cumplir con los siguientes requisitos básicos:

- Disponer de un navegador web actualizado. Durante el desarrollo se ha trabajado y probado principalmente con navegadores basados en Chromium y con Firefox.
- Tener conexión a Internet si se utiliza una versión desplegada de la aplicación.
- Contar con conocimientos básicos sobre diagramas Entidad-Relación y transformación al modelo relacional.
- Disponer de un dispositivo con pantalla y ratón, panel táctil o mecanismo equivalente que permita interactuar con el lienzo gráfico.

Aunque la aplicación ofrece ayuda contextual y mensajes de validación, es recomendable que el usuario conozca previamente los elementos principales del modelo E/R. En particular, resulta útil entender la diferencia entre entidades fuertes y débiles, atributos simples, compuestos o multivaluados, relaciones binarias y ternarias, cardinalidades y claves primarias.

E.3. Instalación

La aplicación puede utilizarse desde una versión desplegada en línea mediante Vercel. En ese caso, no es necesario realizar ninguna instalación local, ya que basta con acceder desde el navegador web.

No obstante, también es posible ejecutar el proyecto en un entorno local. Esta opción resulta útil para revisar el código, realizar pruebas o continuar el desarrollo de la aplicación. Para ello, es necesario disponer previamente de Node.js y npm instalados en el equipo.

En primer lugar, se debe clonar el repositorio del proyecto:

```
git clone <url-del-repositorio>
```

Después, se accede al directorio del proyecto:

```
cd <directorio-del-proyecto>
```

A continuación, se instalan las dependencias:

```
npm install
```

Una vez instaladas, la aplicación puede ejecutarse en modo desarrollo mediante:

```
npm start
```

Tras arrancar el servidor de desarrollo, la aplicación estará disponible normalmente en la dirección local indicada por la propia terminal, habitualmente:

```
http://localhost:3000
```

También es posible generar una versión preparada para producción mediante:

```
npm run build
```

Este último comando crea una versión optimizada de la aplicación en la carpeta de construcción correspondiente, pensada principalmente para despliegue o revisión técnica.

E.4. Manual del usuario

En esta sección se describe el uso principal de la aplicación desde el punto de vista del usuario. El manual se organiza siguiendo un flujo habitual de trabajo: acceso a la aplicación, reconocimiento de la interfaz, creación de elementos del diagrama, edición mediante acciones contextuales, validación, generación de SQL e importación o exportación de información.

Pantalla principal

Al acceder a la aplicación se muestra la pantalla principal del editor, formada por un panel lateral de herramientas y acciones, y un lienzo central donde se construye el modelo Entidad-Relación. La interfaz está pensada para que el usuario pueda añadir elementos de forma progresiva y editar el diagrama directamente sobre el lienzo.

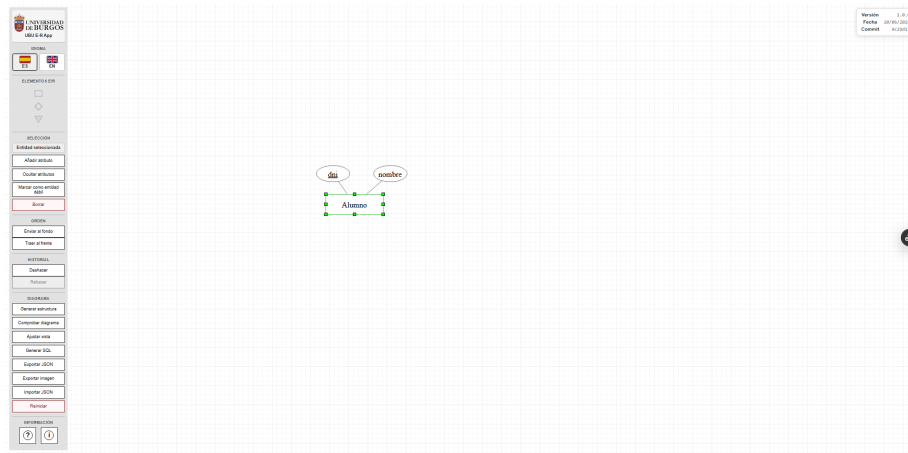


Figura E.1: Pantalla principal de UBU E-R App con un elemento seleccionado.

La zona principal de trabajo es el lienzo. Sobre él se colocan las entidades, atributos, relaciones y otros elementos del modelo. El usuario puede seleccionar los elementos ya creados para moverlos, modificarlos o aplicar acciones contextuales. Cuando no hay ningún elemento seleccionado, la aplicación muestra una guía de ayuda inicial con indicaciones básicas de uso.

En el panel lateral se encuentran las acciones generales de la aplicación, como la importación y exportación de diagramas, la generación de SQL, el ajuste de vista, el cambio de idioma y el acceso a la ayuda o a la información del proyecto. Estas acciones permiten trabajar con el diagrama completo y no con un elemento concreto.

Componentes de la interfaz

La interfaz de la aplicación se divide en varias zonas principales. Cada una de ellas cumple una función concreta dentro del proceso de creación y revisión del diagrama E/R.

En la figura se muestra la interfaz con una entidad seleccionada, lo que permite observar tanto el lienzo de edición como las acciones disponibles en el panel lateral.

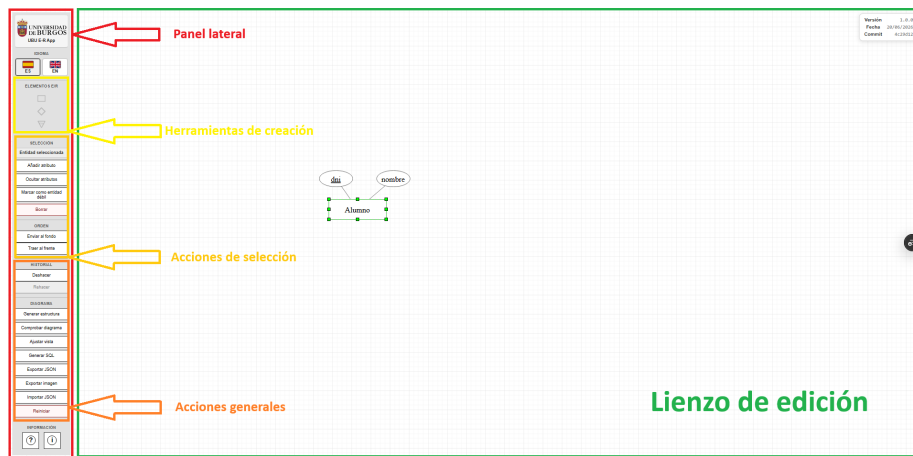


Figura E.2: Componentes principales de la interfaz de usuario.

Los componentes más relevantes son los siguientes:

- **Panel lateral:** agrupa el acceso al cambio de idioma, las herramientas de creación, las acciones contextuales, las acciones generales del diagrama y los botones de información.
- **Herramientas de creación:** permiten crear los elementos principales que se arrastran inicialmente al lienzo, como entidades, relaciones o elementos ISA. Otros elementos, como los atributos, se añaden posteriormente mediante acciones contextuales.
- **Lienzo de edición:** es la zona central en la que se construye el diagrama. Los elementos pueden colocarse, seleccionarse, moverse y relacionarse visualmente.
- **Acciones contextuales:** aparecen al seleccionar un elemento del diagrama. Estas acciones dependen del tipo de elemento seleccionado y permiten modificarlo, añadir elementos asociados o eliminarlo.
- **Acciones generales:** permiten trabajar con el diagrama completo, por ejemplo generar la estructura interna, comprobar el diagrama, ajustar la vista, generar SQL, exportar o importar información y reiniciar el lienzo.
- **Diálogos y salidas de información:** se utilizan para mostrar resultados o información generada por la aplicación, como diagnósticos de validación, mensajes de confirmación o el script SQL resultante.

Creación de elementos del diagrama

La creación de un diagrama comienza incorporando elementos al lienzo de edición. Para ello, el usuario puede utilizar las herramientas de creación disponibles en el panel lateral. Estos elementos permiten construir de forma progresiva el modelo Entidad-Relación.

El funcionamiento básico consiste en arrastrar el elemento deseado desde el panel lateral hasta el lienzo. Una vez situado en el área de trabajo, el elemento pasa a formar parte del diagrama y puede moverse, seleccionarse o modificarse mediante las acciones contextuales correspondientes.

Los elementos principales que se pueden crear desde la interfaz son los siguientes:

- **Entidades:** representan los conjuntos de entidades del modelo. Se muestran mediante rectángulos y pueden tener atributos asociados.
- **Relaciones:** representan interrelaciones entre entidades. Se muestran mediante rombos y posteriormente pueden configurarse indicando sus participantes y cardinalidades.
- **ISA:** permite representar una generalización/especialización de forma inicial y controlada, vinculando una entidad general con una o varias entidades especializadas.

Una forma habitual de comenzar un diagrama es crear una entidad fuerte y añadirle sus atributos principales. Por ejemplo, en una entidad *Alumno* pueden añadirse atributos como *dni* y *nombre*, marcando como clave primaria aquel atributo que identifica de forma única a cada alumno.

A partir de estos elementos básicos, el usuario puede ir completando el diagrama mediante relaciones entre entidades, atributos asociados a entidades o relaciones, entidades débiles cuando sea necesario y estructuras ISA en los casos soportados por la aplicación.

Edición mediante acciones contextuales

La edición de los elementos del diagrama se realiza principalmente mediante la selección directa sobre el lienzo. Al seleccionar una entidad, atributo, relación u otro elemento, el panel lateral muestra las acciones disponibles para ese tipo de elemento. De esta forma, la aplicación evita

mostrar todas las operaciones al mismo tiempo y adapta la interfaz al contexto de trabajo.

En el caso de una entidad, las acciones contextuales permiten añadir atributos, ocultar o mostrar sus atributos asociados, marcarla como entidad débil cuando proceda, eliminarla o modificar su orden visual respecto al resto de elementos del lienzo. Estas acciones facilitan la construcción progresiva del diagrama sin necesidad de utilizar menús adicionales.

En el caso de los atributos, las acciones disponibles dependen del tipo de atributo seleccionado. Entre otras operaciones, el usuario puede marcar un atributo como clave primaria, convertirlo en multivaluado, crear atributos compuestos mediante subatributos o eliminarlo del diagrama.

Las relaciones también disponen de acciones específicas. El usuario puede configurarlas indicando las entidades participantes, ajustar las cardinalidades de sus extremos y, en los casos soportados, añadir atributos propios de la relación. Estas operaciones son especialmente importantes antes de validar el diagrama o generar el SQL, ya que la transformación relacional depende de la información definida en las relaciones.

En el caso de los elementos ISA, las acciones contextuales permiten configurar la generalización y sus especializaciones dentro del soporte inicial incorporado en la aplicación. Este soporte está pensado para representar casos controlados de herencia, en los que existe una entidad general y una o varias especializaciones que heredan su clave.

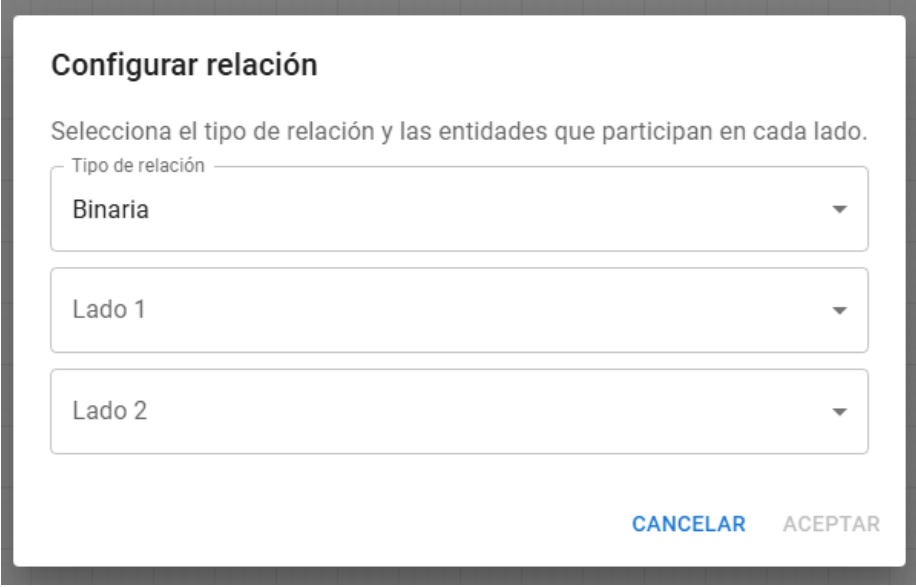
Cuando se seleccionan varios elementos al mismo tiempo, la interfaz muestra la selección múltiple de forma visual y permite realizar operaciones conjuntas, como mover o eliminar los elementos seleccionados. En este caso, las acciones individuales se ocultan para evitar aplicar operaciones específicas sobre un conjunto de elementos que pueden ser de tipos distintos.

Configuración de relaciones y cardinalidades

Las relaciones se incorporan al diagrama arrastrando el elemento correspondiente desde el panel lateral hasta el lienzo. Una vez creada la relación, es necesario configurarla para indicar qué entidades participan en ella y qué cardinalidades se aplican en cada extremo.

Para configurar una relación, el usuario debe seleccionarla en el lienzo y utilizar las acciones contextuales disponibles en el panel lateral. Desde estas acciones puede abrirse el diálogo de configuración de la relación, donde se definen sus participantes. En las relaciones binarias se seleccionan las

dos entidades conectadas por la relación. En los casos soportados, como las relaciones ternarias, se pueden definir más participantes.

El formulario tiene un título "Configurar relación" en negrita. Debajo, una instrucción dice "Selecciona el tipo de relación y las entidades que participan en cada lado.". Hay tres campos de selección: "Tipo de relación" con el valor "Binaria" seleccionado, "Lado 1" y "Lado 2", todos con flechas hacia abajo. En la parte inferior derecha, hay dos botones: "CANCELAR" en azul y "ACEPTAR" en gris.

Configurar relación

Selecciona el tipo de relación y las entidades que participan en cada lado.

Tipo de relación —

Binaria ▼

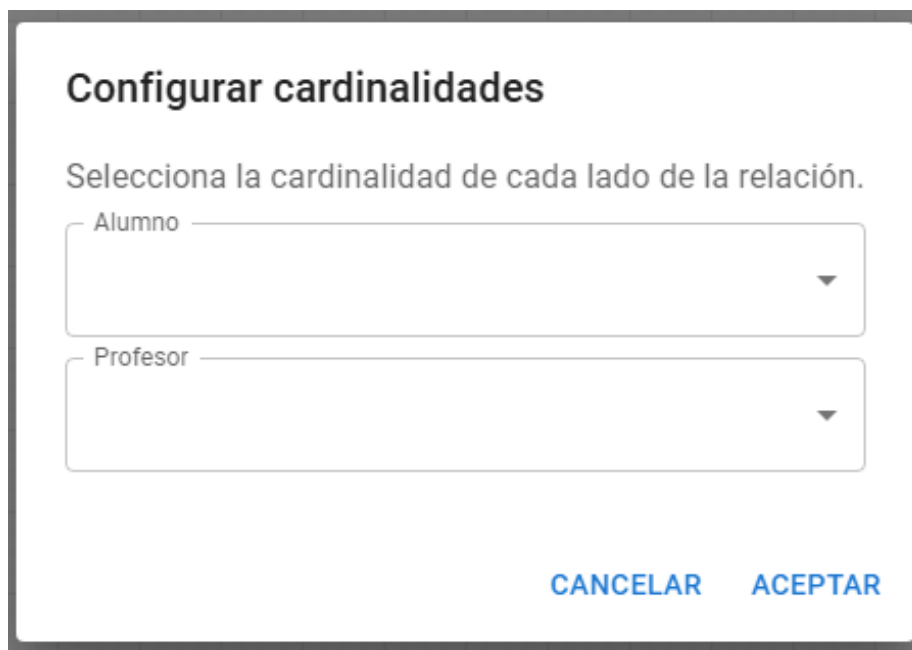
Lado 1 ▼

Lado 2 ▼

CANCELAR ACEPTAR

Figura E.3: Configuración de los participantes de una relación.

Una vez definidos los participantes, la relación puede completarse indicando las cardinalidades correspondientes. Estas cardinalidades representan las restricciones de participación entre las entidades y la relación, y son necesarias para que la aplicación pueda validar correctamente el diagrama y transformarlo después al modelo relacional.



Configurar cardinalidades

Selecciona la cardinalidad de cada lado de la relación.

Alumno

Profesor

CANCELAR **ACEPTAR**

Figura E.4: Configuración de cardinalidades de una relación.

La aplicación permite trabajar con cardinalidades habituales del modelo E/R, como relaciones uno a uno, uno a muchos y muchos a muchos. La información introducida en este paso influye directamente en la estructura relacional generada posteriormente. Por ejemplo, una relación muchos a muchos se transforma normalmente en una tabla intermedia, mientras que una relación uno a muchos suele traducirse mediante una clave ajena en la tabla correspondiente.

En las relaciones que admiten atributos propios, el usuario puede añadir dichos atributos desde las acciones contextuales de la relación. Esta opción resulta especialmente útil en relaciones muchos a muchos, cuando la relación no solo conecta entidades, sino que también tiene información propia que debe conservarse en el modelo relacional.

Cuando una relación todavía no ha sido configurada correctamente, la aplicación puede mostrar mensajes de validación o impedir determinadas operaciones hasta que la información necesaria esté completa. De esta forma se reduce la posibilidad de generar una estructura relacional incoherente o incompleta.

Validación del diagrama y generación de SQL

Una vez creado el diagrama, la aplicación permite comprobar su coherencia antes de generar el script SQL. Esta validación ayuda a detectar errores o situaciones incompletas, como entidades sin clave primaria, relaciones sin configurar, nombres repetidos o cardinalidades no definidas correctamente.

Para realizar esta comprobación, el usuario puede utilizar la acción de validación disponible en el panel lateral. Si el diagrama contiene algún problema, la aplicación muestra un diagnóstico con los aspectos que deben revisarse. De esta forma, el usuario puede corregir el modelo antes de continuar con la transformación relacional.

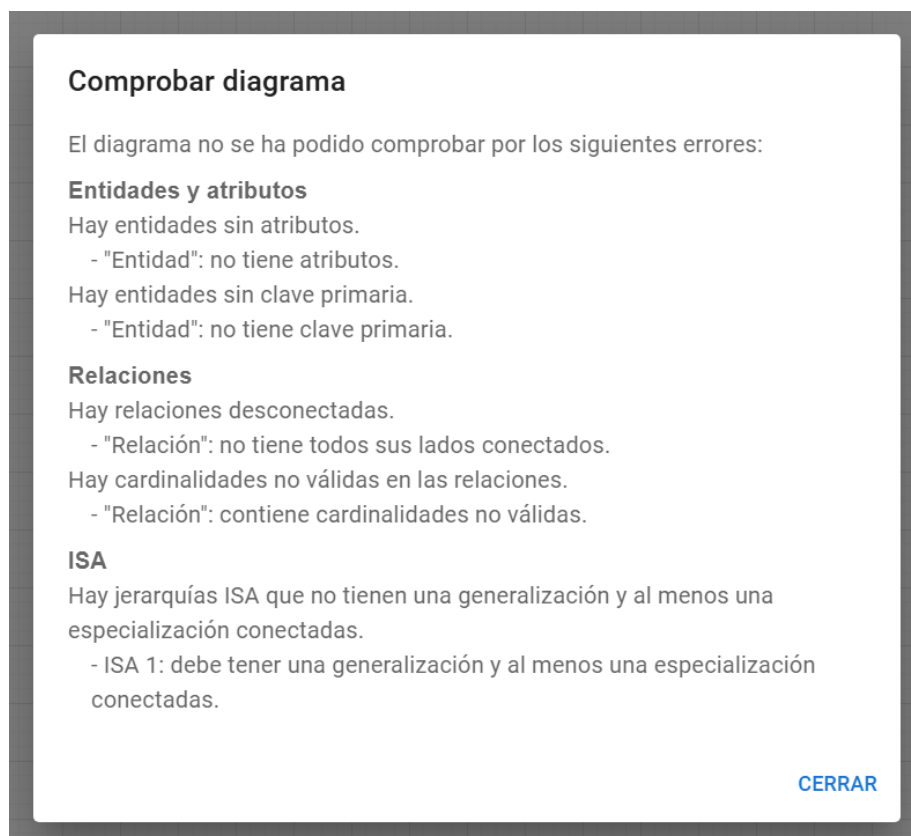


Figura E.5: Resultado de la validación de un diagrama.

Cuando el diagrama es válido, el usuario puede generar el SQL mediante la acción correspondiente. La aplicación transforma internamente el modelo Entidad-Relación en una estructura relacional y, a partir de ella, construye un script SQL con las tablas, claves primarias y claves ajenas necesarias.

```
DROP TABLE IF EXISTS Entidad, Entidad_1, Relacion CASCADE;

CREATE TABLE Entidad (
  id VARCHAR(40) PRIMARY KEY,
  Atributo_1 VARCHAR(40)
);

CREATE TABLE Entidad_1 (
  id VARCHAR(40) PRIMARY KEY,
  Atributo_1 VARCHAR(40)
);

CREATE TABLE Relacion (
  id_Relacion_1 VARCHAR(40) REFERENCES Entidad,
  id_Relacion_2 VARCHAR(40) REFERENCES Entidad_1,
  Atributo VARCHAR(40),
  PRIMARY KEY (id_Relacion_1, id_Relacion_2)
);
```

Figura E.6: Script SQL generado a partir del diagrama.

El SQL generado se presenta en un formato simplificado para facilitar su lectura. En lugar de mostrar una salida excesivamente fragmentada, la aplicación agrupa la información principal de cada tabla y mantiene las restricciones relevantes para representar el modelo relacional obtenido. Este resultado puede utilizarse como referencia para comprender cómo se traduce el diagrama al modelo relacional o como punto de partida para su adaptación a un sistema gestor de bases de datos concreto.

La transformación depende de los elementos definidos en el diagrama. Las entidades fuertes se convierten en tablas, las claves primarias se reflejan como restricciones de identificación y las relaciones se transforman según sus cardinalidades. Por ejemplo, las relaciones muchos a muchos generan una tabla intermedia, mientras que las relaciones uno a muchos se representan habitualmente mediante claves ajenas.

En el caso de las entidades débiles y de las estructuras ISA soportadas, la aplicación aplica reglas específicas dentro del alcance implementado. Las entidades débiles se transforman teniendo en cuenta su dependencia de identificación, mientras que las estructuras ISA se tratan como un soporte

inicial y controlado en el que las especializaciones heredan la clave de la entidad general.

La generación de SQL debe entenderse como una ayuda para obtener una representación relacional coherente con el diagrama construido. En caso de que el usuario necesite un script adaptado a un entorno concreto, puede revisar y ajustar posteriormente el resultado generado.

Importación y exportación de diagramas

La aplicación permite guardar y recuperar diagramas mediante ficheros JSON. Esta funcionalidad resulta útil para conservar el trabajo realizado, compartir un diagrama con otra persona o continuar editándolo en otro momento.

Para guardar el diagrama actual, el usuario puede utilizar la acción de exportación JSON disponible en el panel lateral. Al seleccionarla, la aplicación genera un fichero con la información interna del diagrama, incluyendo los elementos creados, sus posiciones y la configuración necesaria para reconstruirlo posteriormente.

Para cargar un diagrama previamente exportado, se utiliza la acción de importación JSON. Al importar un fichero, la aplicación ofrece un comportamiento controlado para decidir cómo incorporar el contenido al lienzo. En función de la opción seleccionada, el diagrama importado puede reemplazar al diagrama actual o combinarse con los elementos ya existentes.



Importar diagrama desde JSON

Selecciona un archivo JSON exportado previamente y elige si quieres reemplazar el diagrama actual o combinarlo con él.

Modo
Reemplazar el diagrama actual

Seleccionar archivo Ningún archivo seleccionado

CERRAR

Figura E.7: Importación de un diagrama desde un fichero JSON.

La opción de reemplazar resulta adecuada cuando se desea abrir un diagrama guardado y trabajar únicamente sobre él. En cambio, la opción de

combinar permite añadir los elementos importados al diagrama que ya está abierto, lo que puede ser útil para reutilizar partes de otros modelos.

Además de la exportación JSON, la aplicación permite exportar una representación visual del diagrama. Esta opción genera una imagen del lienzo en formatos como PNG o SVG, pensada para incluir el diagrama en documentos, entregas o material de apoyo. A diferencia del JSON, estos formatos visuales no están pensados para volver a editar el diagrama dentro de la aplicación, sino para obtener una copia gráfica del resultado.

Acciones generales del diagrama

El panel lateral incluye varias acciones que afectan al diagrama completo. Estas acciones sirven para comprobar, transformar, ajustar, guardar o reiniciar el trabajo realizado en el lienzo.

Entre las acciones generales más relevantes se encuentran las siguientes:

- **Generar estructura:** permite crear una estructura E/R básica de ejemplo y elegir si reemplaza el diagrama actual o se combina con el contenido existente. Esta acción resulta útil como apoyo para probar la herramienta o iniciar rápidamente un modelo sencillo.
- **Comprobar diagrama:** ejecuta la validación del modelo y muestra los posibles problemas detectados antes de generar SQL.
- **Ajustar vista:** centra y adapta la vista del lienzo para facilitar la visualización del diagrama completo.
- **Generar SQL:** transforma el diagrama validado en una representación relacional y genera el script SQL correspondiente.
- **Exportar JSON:** guarda el diagrama en un fichero que puede importarse posteriormente en la aplicación.
- **Importar JSON:** carga un diagrama desde un fichero externo, permitiendo reemplazar el contenido actual o combinarlo con el diagrama abierto.
- **Exportar visualmente:** genera una imagen del diagrama en formato PNG o SVG.
- **Reiniciar:** elimina el contenido actual del lienzo tras solicitar confirmación al usuario.

Estas acciones permiten gestionar el ciclo completo de trabajo con un diagrama: crear el modelo, revisarlo, obtener su representación SQL, guardarlo, recuperarlo o exportarlo como imagen. En las operaciones destructivas, como el reinicio del lienzo o la importación con reemplazo, la aplicación solicita confirmación para evitar pérdidas accidentales de información.

Selección múltiple y edición conjunta

Además de seleccionar elementos de forma individual, la aplicación permite seleccionar varios elementos del lienzo al mismo tiempo. Esta funcionalidad resulta útil cuando el usuario quiere reorganizar una parte del diagrama o eliminar varios elementos en una única operación.

La selección múltiple se muestra visualmente sobre el lienzo, permitiendo identificar qué elementos forman parte del conjunto seleccionado. Una vez seleccionados, los elementos pueden moverse de forma conjunta, manteniendo su disposición relativa dentro del diagrama.

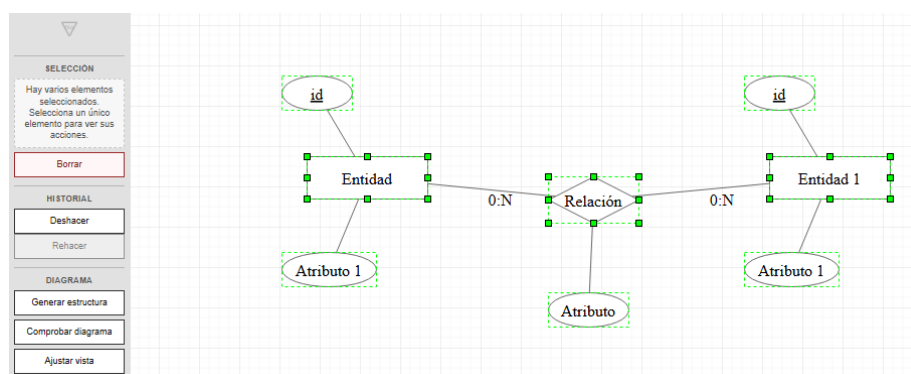


Figura E.8: Selección múltiple de elementos en el lienzo.

Cuando existe una selección múltiple, la interfaz oculta las acciones contextuales individuales. Esta decisión evita que el usuario aplique por error una acción específica de una entidad, atributo o relación sobre un conjunto de elementos de distintos tipos. En su lugar, se mantienen disponibles únicamente las operaciones que tienen sentido sobre el conjunto seleccionado, como el movimiento conjunto o la eliminación.

La eliminación de varios elementos seleccionados permite limpiar partes del diagrama de forma más cómoda. En las operaciones destructivas de mayor alcance, como el reinicio del lienzo o la importación con reemplazo, la aplicación utiliza confirmaciones o mecanismos de control para reducir la posibilidad de pérdidas accidentales de información.

Ayuda, idioma e información del proyecto

La aplicación incluye varias acciones de apoyo pensadas para facilitar su uso y proporcionar información adicional al usuario. Estas acciones no modifican directamente el modelo E/R, pero ayudan a comprender la interfaz, consultar información general del proyecto o adaptar el idioma de la aplicación.

El selector de idioma permite alternar entre español e inglés. Al cambiar el idioma, la interfaz actualiza sus textos principales, incluyendo botones, mensajes de ayuda y diagnósticos mostrados al usuario.

La acción de ayuda muestra una guía breve sobre el funcionamiento general de la aplicación. Esta guía sirve como referencia rápida para recordar cómo crear elementos, utilizar las acciones contextuales, validar el diagrama o generar SQL.

La opción de información del proyecto muestra datos generales sobre la aplicación, incluyendo su nombre, autoría, créditos, tecnologías utilizadas, referencia al proyecto original y licencia de la versión actual.



Figura E.9: Información general del proyecto.

Estas acciones complementan el flujo principal de edición y contribuyen a que la aplicación sea más comprensible para usuarios que acceden por primera vez al editor o que necesitan recordar el propósito de alguna funcionalidad.

Flujo de trabajo recomendado

Aunque la aplicación permite editar el diagrama de forma flexible, se recomienda seguir un flujo de trabajo ordenado para reducir errores y facilitar la generación posterior del SQL.

En primer lugar, conviene crear las entidades principales del modelo y añadir sus atributos más relevantes, marcando como clave primaria aquellos atributos que identifiquen a cada entidad. A continuación, pueden incorporarse las relaciones entre entidades, configurando sus participantes y cardinalidades. En caso de utilizar entidades débiles o estructuras ISA, es recomendable comprobar que su configuración queda correctamente definida antes de continuar con el resto del diagrama.

Una vez construido el modelo, se aconseja utilizar la acción de comprobación del diagrama. Esta validación permite detectar problemas antes de generar la estructura relacional o el script SQL. Si aparecen mensajes de diagnóstico, el usuario debe revisar los elementos indicados y corregir la información incompleta o incoherente.

Cuando el diagrama sea válido, puede generarse el SQL y revisarse el resultado obtenido. Si el modelo se quiere conservar para futuras ediciones, debe exportarse en formato JSON. Si únicamente se necesita una representación visual para incluirla en un documento o entrega, puede utilizarse la exportación en formato PNG o SVG.

De forma resumida, el flujo habitual de uso es el siguiente:

1. Crear las entidades principales.
2. Añadir atributos y definir claves primarias.
3. Incorporar relaciones y configurar sus participantes.
4. Definir las cardinalidades correspondientes.
5. Añadir elementos adicionales, como entidades débiles, atributos de relación o estructuras ISA, cuando proceda.
6. Comprobar el diagrama mediante la validación.
7. Generar y revisar el SQL.
8. Exportar el diagrama en JSON o como imagen, según el uso previsto.

Este flujo no es obligatorio, pero ayuda a mantener el diagrama coherente y facilita que la transformación relacional refleje correctamente el modelo diseñado.

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluye una reflexión personal sobre los aspectos de sostenibilidad abordados durante el desarrollo del Trabajo Fin de Grado. En este contexto, la sostenibilidad no se entiende únicamente desde una perspectiva medioambiental, sino también desde dimensiones educativas, técnicas, sociales y profesionales.

El proyecto desarrollado consiste en la evolución de una aplicación web para la creación, validación y transformación de diagramas Entidad-Relación. Por ello, su relación con la sostenibilidad se sitúa principalmente en el ámbito de la educación, la reutilización de software, la documentación, el mantenimiento responsable y el acceso a herramientas digitales de apoyo al aprendizaje.

A continuación, se relacionan algunas de las competencias trabajadas durante el proyecto con distintos Objetivos de Desarrollo Sostenible de la Agenda 2030 de Naciones Unidas [2].

F.2. Competencias de sostenibilidad adquiridas

ODS 4: Educación de calidad

La aplicación desarrollada puede servir como apoyo para estudiantes que estén aprendiendo el modelo Entidad-Relación y su transformación al modelo relacional. El hecho de poder crear diagramas, validarlos y observar una posible generación de SQL facilita la práctica con ejemplos vistos en clase y permite detectar errores de forma más visual.

Además, al tratarse de una aplicación web, su uso no requiere instalar herramientas complejas. Esto reduce la barrera de entrada para el estudiante y permite centrarse en el aprendizaje del modelo. No obstante, la herramienta debe entenderse como un complemento al estudio y a la explicación del profesorado, no como un sustituto del razonamiento teórico necesario para diseñar correctamente una base de datos.

ODS 9: Industria, innovación e infraestructura

El proyecto se ha basado en la evolución de una herramienta software existente, incorporando nuevas funcionalidades y estabilizando su comportamiento. Esta forma de trabajo se aproxima a situaciones habituales en el ámbito profesional, donde no siempre se parte de cero, sino que es necesario comprender, mantener y ampliar sistemas ya desarrollados.

Durante el trabajo se ha seguido una organización progresiva mediante issues, hitos, pruebas y documentación. Esta metodología ha permitido abordar los cambios de forma controlada, reduciendo el riesgo de introducir modificaciones incoherentes con el resto del sistema. Desde este punto de vista, el proyecto contribuye al desarrollo de una infraestructura digital académica más mantenible y preparada para futuras ampliaciones.

ODS 12: Producción y consumo responsables

Uno de los aprendizajes más importantes del proyecto ha sido comprender que reutilizar software existente también implica responsabilidad. Aunque partir de una aplicación heredada puede parecer más sencillo que comenzar desde cero, en la práctica fue necesario dedicar una parte importante del trabajo a entender su estructura interna, sus decisiones de diseño y las dependencias entre el lienzo, el modelo interno, las validaciones, la transformación relacional y la generación de SQL.

Esta experiencia ha permitido valorar la reutilización como una práctica sostenible, siempre que vaya acompañada de análisis, documentación y mantenimiento. En lugar de descartar el trabajo previo, se ha aprovechado la base existente y se han introducido mejoras de forma progresiva. También se ha prestado atención a la licencia del proyecto actual y a las dependencias utilizadas, con el objetivo de respetar el trabajo de terceros y aclarar las condiciones de uso de la versión desarrollada.

ODS 17: Alianzas para lograr los objetivos

El desarrollo del proyecto ha requerido una comunicación continua con el tutor académico y una revisión progresiva de las decisiones tomadas. Esta colaboración ha sido especialmente importante porque el modelo Entidad-Relación puede admitir distintos matices de interpretación, y la aplicación debía mantenerse alineada con los materiales docentes y con el criterio seguido en la asignatura.

También se ha intentado dejar una base documentada para que futuros estudiantes o desarrolladores puedan comprender el estado del proyecto, sus funcionalidades, sus limitaciones y sus posibles líneas de continuación. En este sentido, la documentación forma parte de la sostenibilidad del trabajo, ya que facilita que el conocimiento generado no quede limitado únicamente al periodo de desarrollo del TFG.

F.3. Reflexión final

La realización de este Trabajo Fin de Grado me ha permitido acercarme a una forma de trabajo más parecida a un contexto profesional real. No se ha tratado únicamente de implementar funcionalidades, sino de comprender un sistema existente, organizar tareas, tomar decisiones, validar cambios y documentar el resultado.

Uno de los aspectos más relevantes ha sido comprobar que la sostenibilidad del software no depende solo de que una aplicación funcione, sino también de que pueda entenderse, mantenerse y evolucionar. En algunos casos, el mayor esfuerzo no estuvo en programar una nueva funcionalidad, sino en comprender por qué el sistema estaba organizado de una determinada manera y cómo introducir cambios sin romper su coherencia.

El proyecto tiene un alcance claramente académico y presenta limitaciones conocidas. No pretende cubrir todos los casos posibles del modelo Entidad-Relación, el soporte para ISA se ha incorporado de forma inicial y controlada, y las agregaciones quedan fuera del alcance actual. Aun así, considero que el trabajo contribuye de forma modesta pero realista a la sostenibilidad educativa y técnica, ofreciendo una herramienta útil para el aprendizaje y dejando una base más estable, documentada y mantenible para futuras ampliaciones.

Bibliografía

- [1] Jesús Maudes Raedo. Tema 6. El modelo E/R y su representación lógica en SQL. Material docente de Bases de Datos, Grado en Ingeniería Informática, Universidad de Burgos, 2017. Material docente consultado durante la realización del trabajo.
- [2] Naciones Unidas. Objetivos de Desarrollo Sostenible. <https://sdgs.un.org/goals>. [Internet; consultado 21-junio-2026].
- [3] Open Source Initiative. The MIT License. <https://opensource.org/license/mit>. [Internet; consultado 21-junio-2026].
- [4] The Apache Software Foundation. Apache License, Version 2.0. <https://www.apache.org/licenses/LICENSE-2.0.txt>. [Internet; consultado 21-junio-2026].